



The QUILCening

The Internet's Biggest Upgrade in Over 50 Years:
Is your network ready? Are YOU ready?

(KY)NUG Lexington | 2026-01-22

Victor Weis
Systems Architect
Fortinet

Agenda

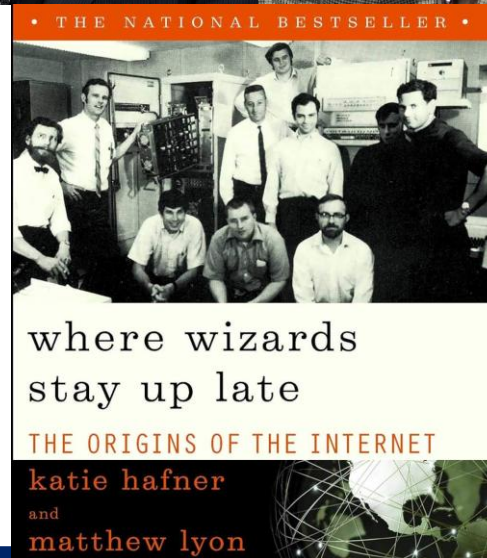
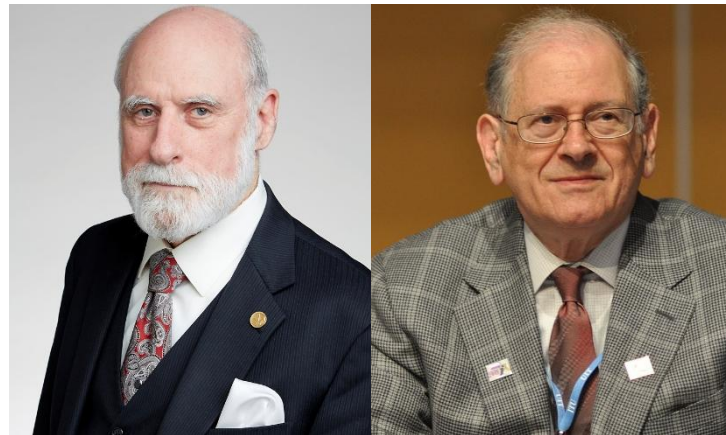


- What is QUIC? What problems does QUIC address?
- How does QUIC impact my business/organization?
- Why do engineers need to know about QUIC:
 - Network Engineer
 - Security Engineers
 - Application Developers
- How rapidly is QUIC being adopted? When will this be important?
- Where can I learn more?

QUIC Working Group Website: <https://quicwg.org/>

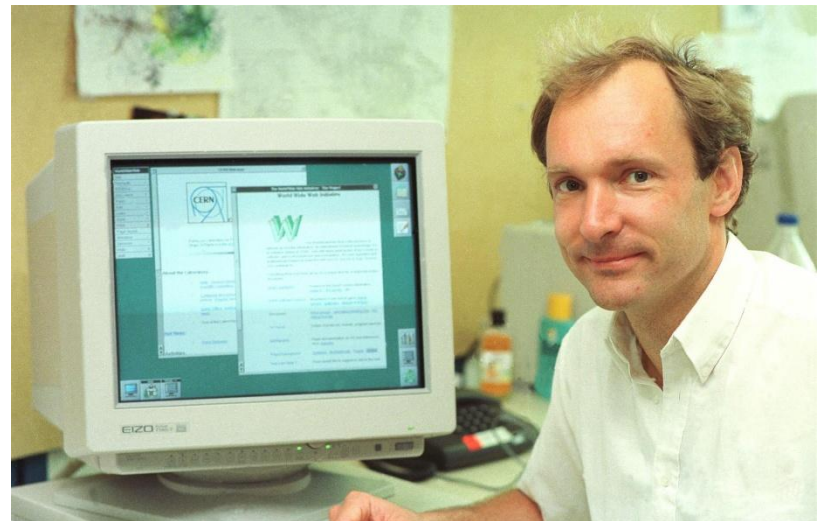
TCP: The Transport Layer OG

- [RFC 675](#) was born in 1974
- This makes TCP over 50 years old!
 - Thank you Vint Cerf and Bob Kahn!
- Dominant since the early 1990's
- MOST of the modern web was built on TCP:
 - HTTP 1.1
 - HTTP2
 - SSL/TLS
 - Etc.
- IP Protocol Number for TCP is 6



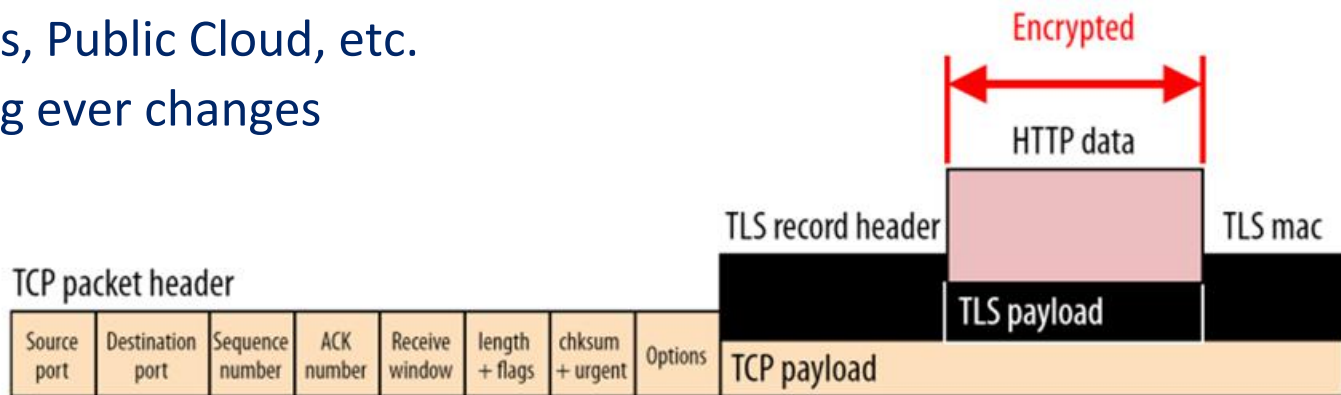
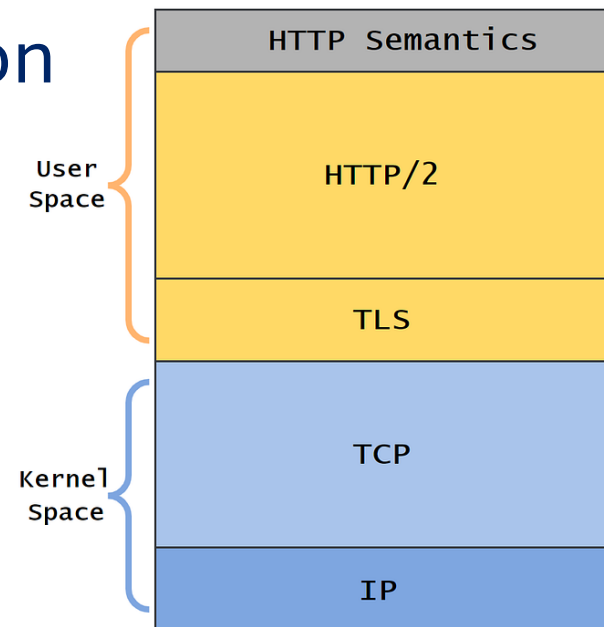
Historical Origins of TCP Limitations

- Designed when main app-layer protocols were telnet, FTP, and SMTP
- Primary use-cases were remote computation, file transfers, and email
- Tim Berners-Lee's "World Wide Web" wouldn't develop until much later
 - HTTP 0.9 published in 1989
 - HTTP 1.0 in 1996 ([RFC 1945](#)).
- TimBL used TCP because it was available
 - not because it was optimal
- Jump-started the web revolution and had a GREAT run
- Now: showing its age; struggling to keep up with the demands of the modern web.



TCP Limitations: Ossification

- Inherently difficult to improve for two reasons
- First: Kernel Space implementation requires cooperation from OS developers
 - Microsoft, Apple, and Linux Foundation all must agree
- Second: Full [wire-image](#) requires cooperation from middlebox manufacturers
 - Cisco, Arista, Palo, Fortinet, etc., all must agree
 - Also: CDNs, Public Cloud, etc.
- Result: Nothing ever changes



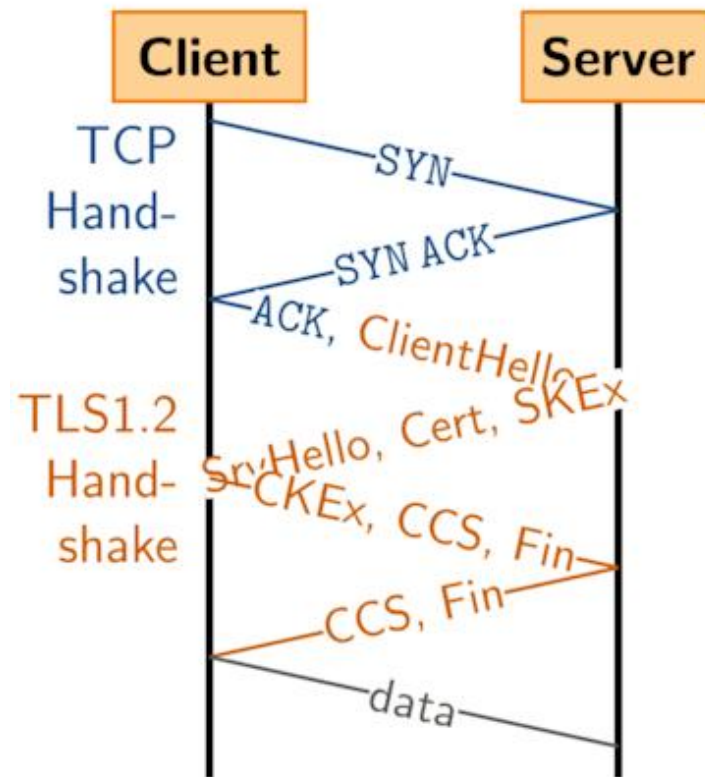
TCP Limitations: Ossification

- Inherently difficult to improve for two reasons
- First: Kernel Space implementation requires cooperation from OS developers
 - Microsoft, Apple, and Linux Foundation all must agree
- Second: Full [wire-image](#) requires cooperation from middlebox manufacturers
 - Cisco, Arista, Palo, Fortinet, etc., all must agree
 - Also: CDNs, Public Cloud, etc.
- Result: Nothing ever changes



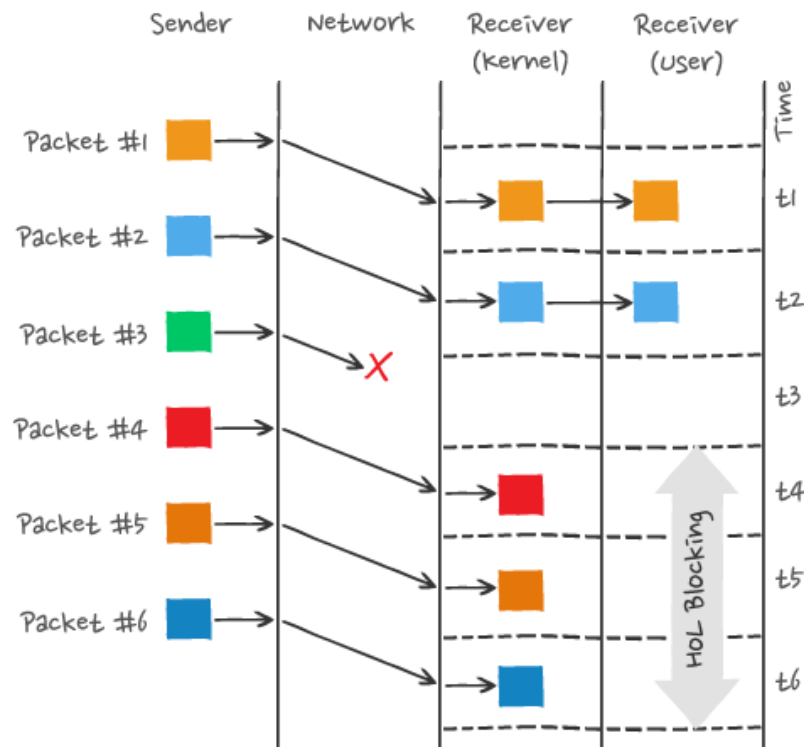
TCP Limitations: Security & Latency

- No inherent security
- SSL/TLS is a bolt-on solution
- TCP 3-way handshake happens first to establish session
- Then TLS handshake happens next to provide encryption
- Extra round-trips = extra latency
- Still leaves TCP headers exposed



TCP Limitations: No Multiplexing

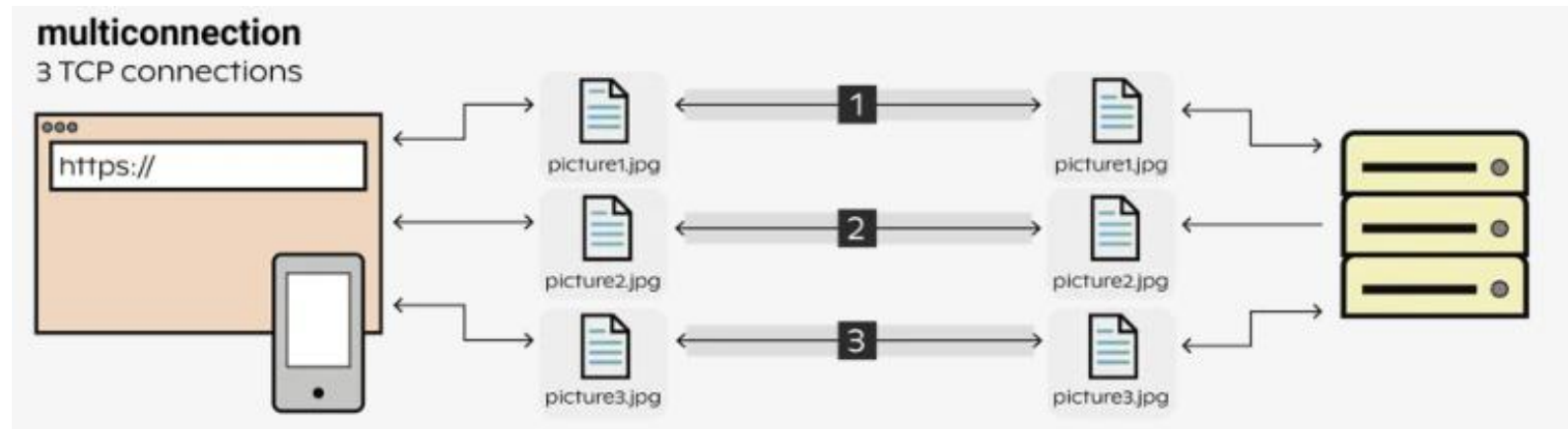
- TCP provide a strictly-ordered, reliable stream of packets
- No native multiplexing capability
- Naturally suffers from head-of-line blocking
- Responses must arrive in order
- Slow responses block following ones
- Browsers busy waiting; web pages/app load slowly



Source: <https://ably.com/topic/http3>

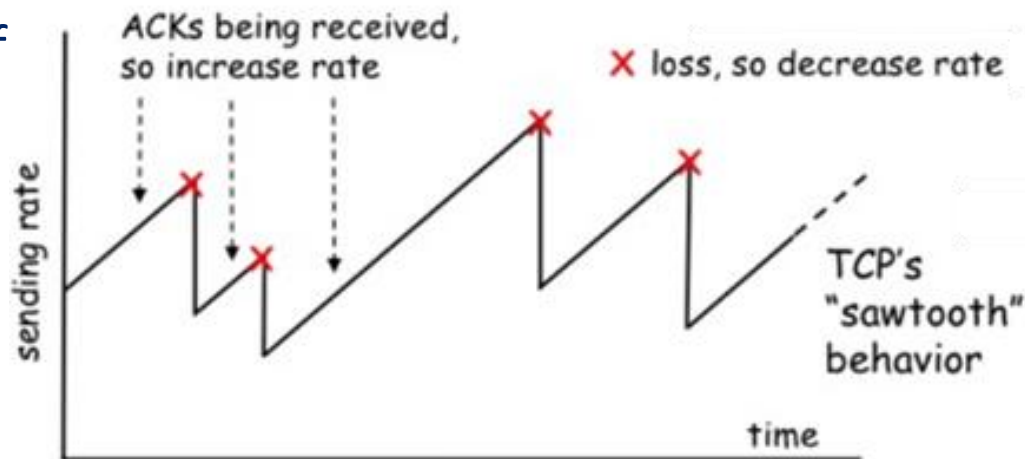
TCP Limitations: No Multiplexing

- Kludge: use multiple simultaneous TCP streams
- Produces a LOT of overhead, and doesn't scale beyond ~6 streams
- Modern web apps need many simultaneous elements
- Other attempts: HTTP/2 and SPDY



TCP Limitations: Passive Congestion Control

- Originally had no explicit, pro-active congestion control
- ECN tried to address this, but slow adoption made it DOA
 - RFC 3168, 2001
 - Real-world adoption still VERY low even in 2025*
- Clients flood a link to death until congestion occurs
- Then retroactively back off
 - aka “Additive Increase Multiplicative Decrease” or AIMD
- Tragedy of the Commons



*Ref: <https://blog.apnic.net/2025/09/11/measuring-explicit-congestion-notification/>

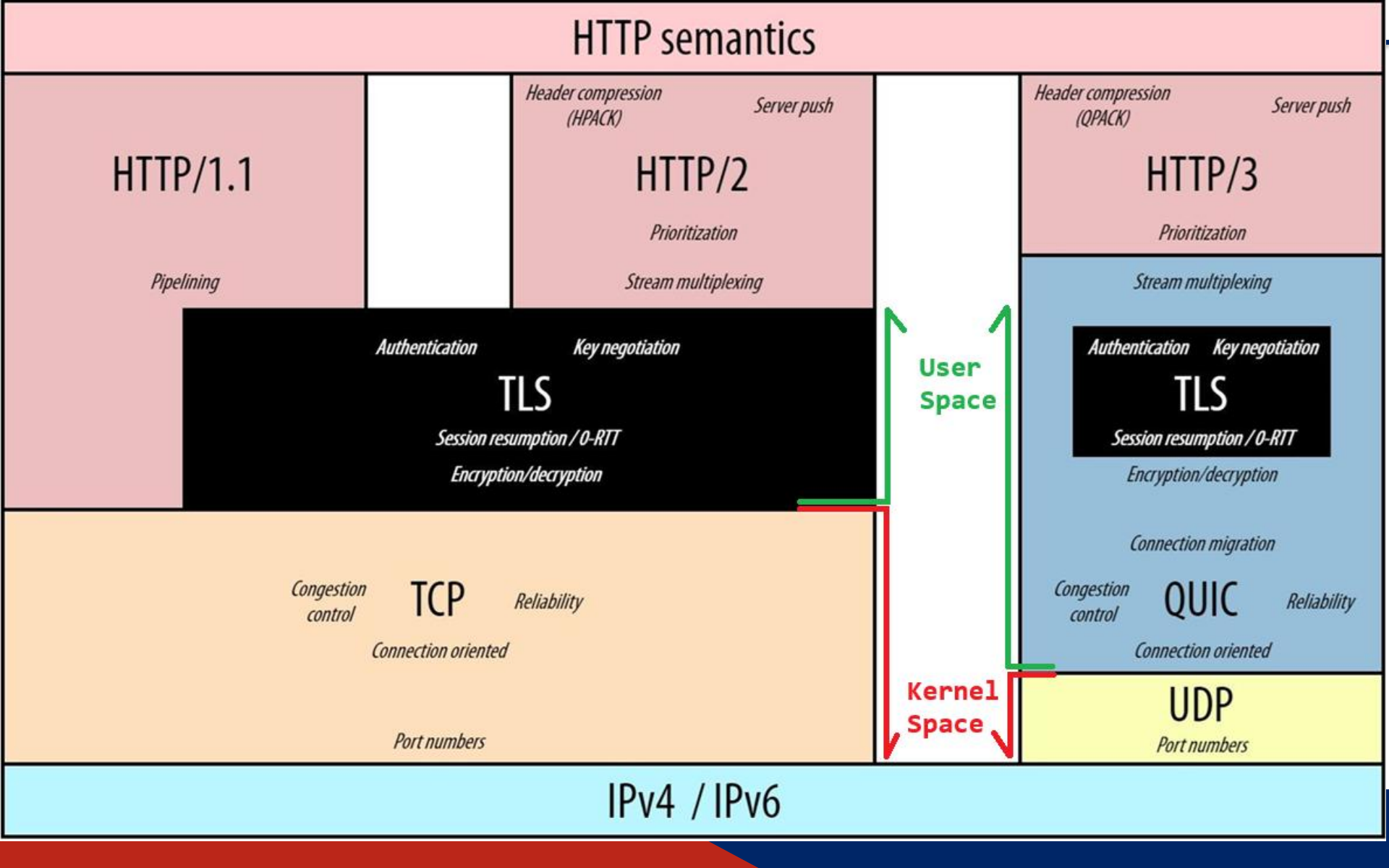
Introducing QUIC



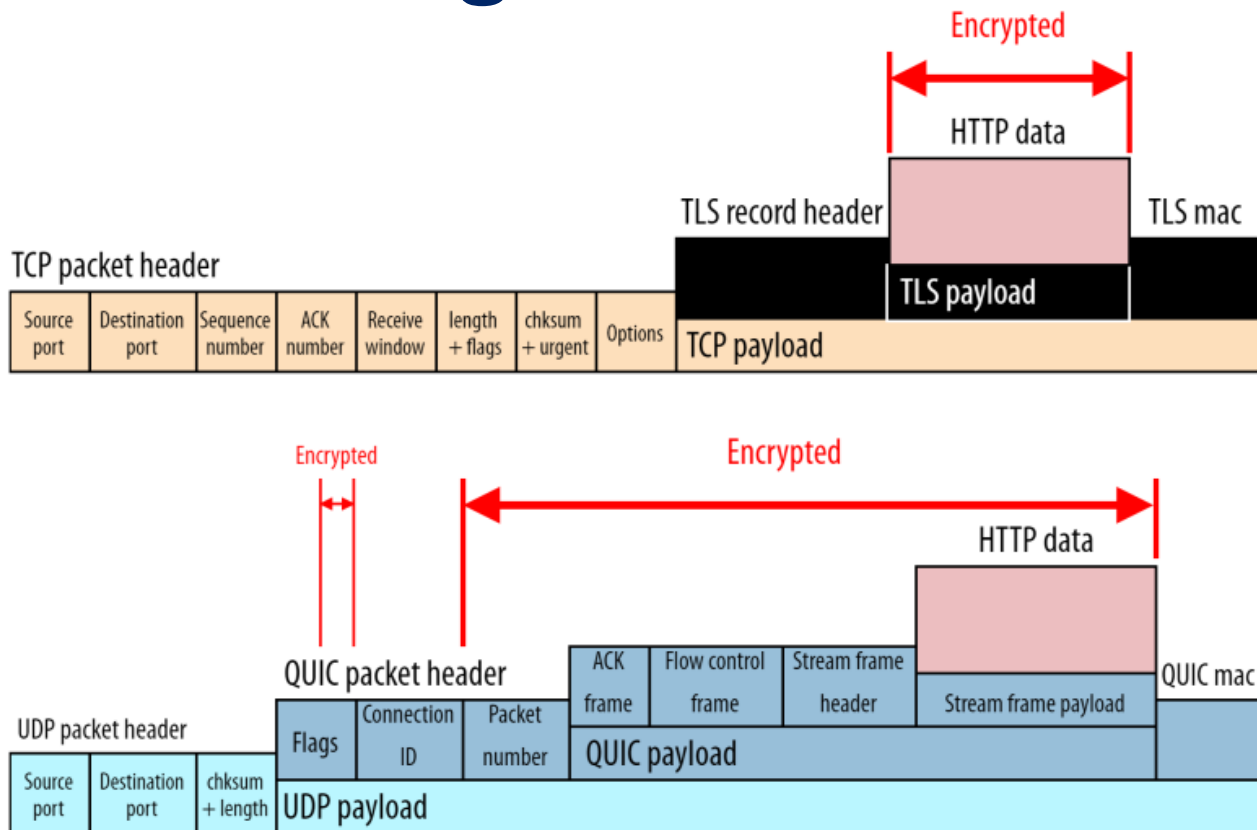
- A new connection-oriented transport-layer protocol
- Informed by 50 years of lessons-learned from TCP
- How to address ossification?
 - Move out of Kernel Space and into User Space
 - Reduce Wire Image as much as possible
- Can't introduce new IP Protocol Number
- Instead, re-use UDP (17) for basic port control only
- Then build NEW connection-oriented, secure-by-default features on TOP of UDP – in User Space

Overview of Main QUIC Innovations

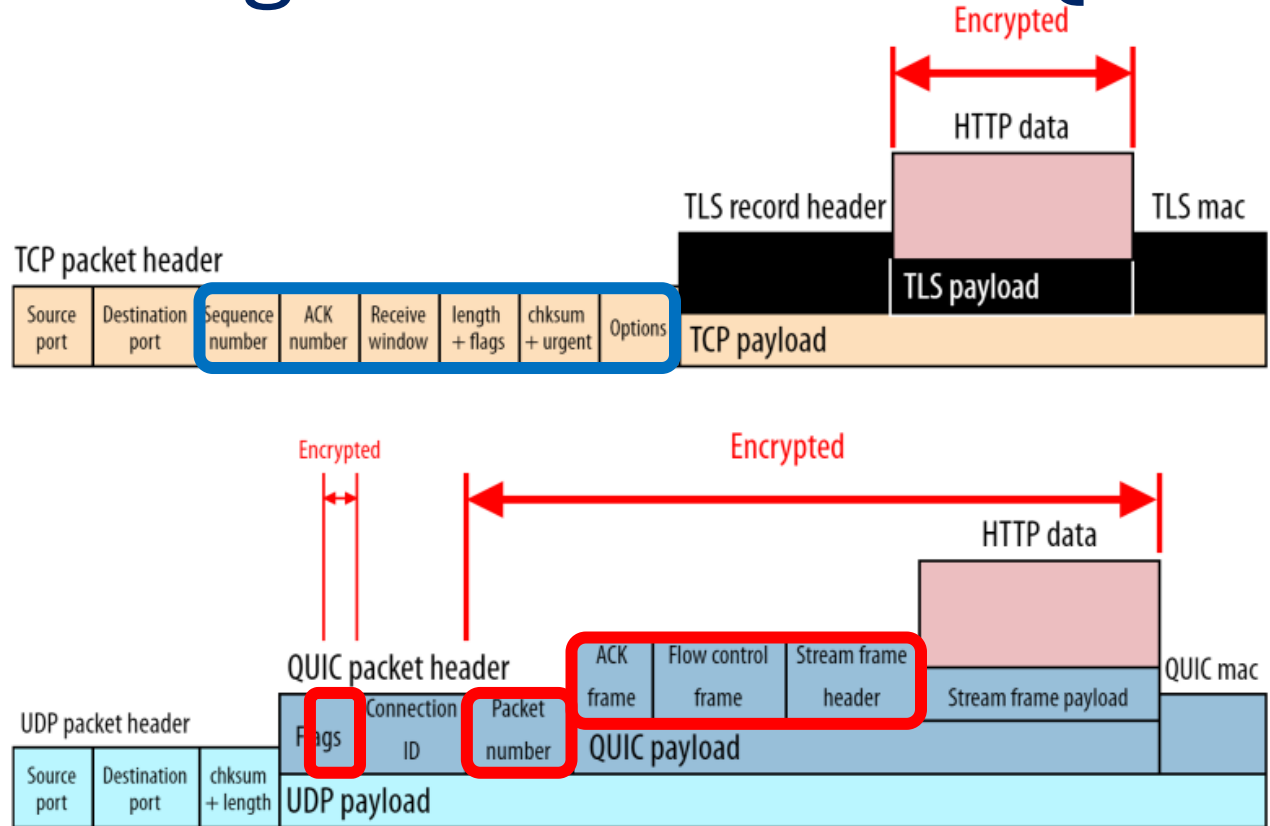
- Less Ossification: User Space implementation + smaller wire-image
- Better Security: More of the packet is encrypted + TLS mandatory
- Reduced Latency: Establish session + encryption in same RTT
- Built-in Explicit Congestion Control with rapid adoption
- Native Multiplexing to eliminate Head-of-Line Blocking
- End results:
 - A faster, more secure, more reliable user experience
 - WWW becomes more agile & less fragile



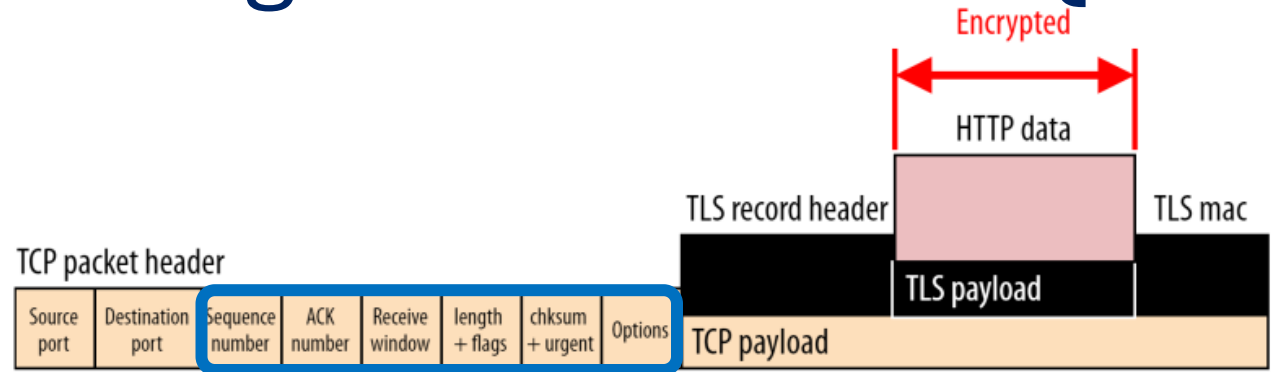
Wire Image of TCP/TLS vs QUIC



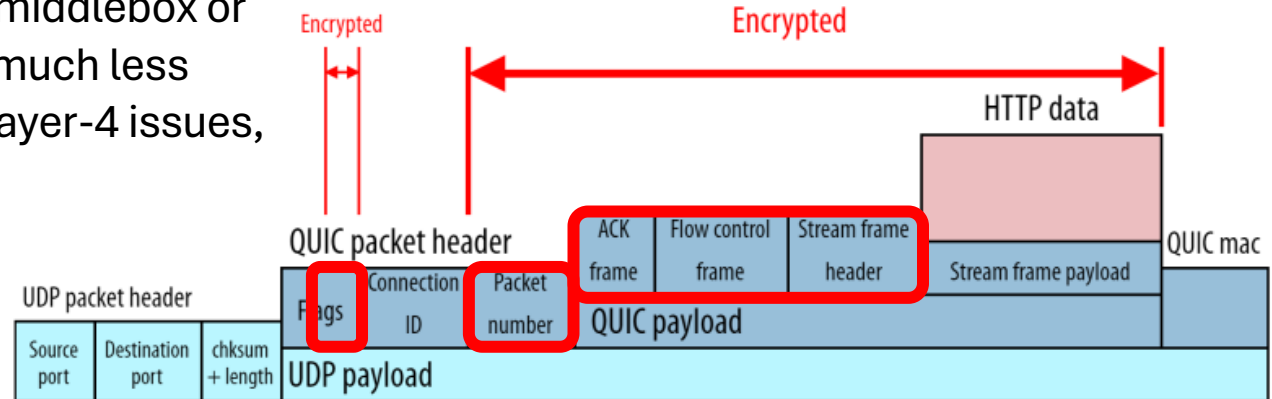
Wire Image of TCP/TLS vs QUIC



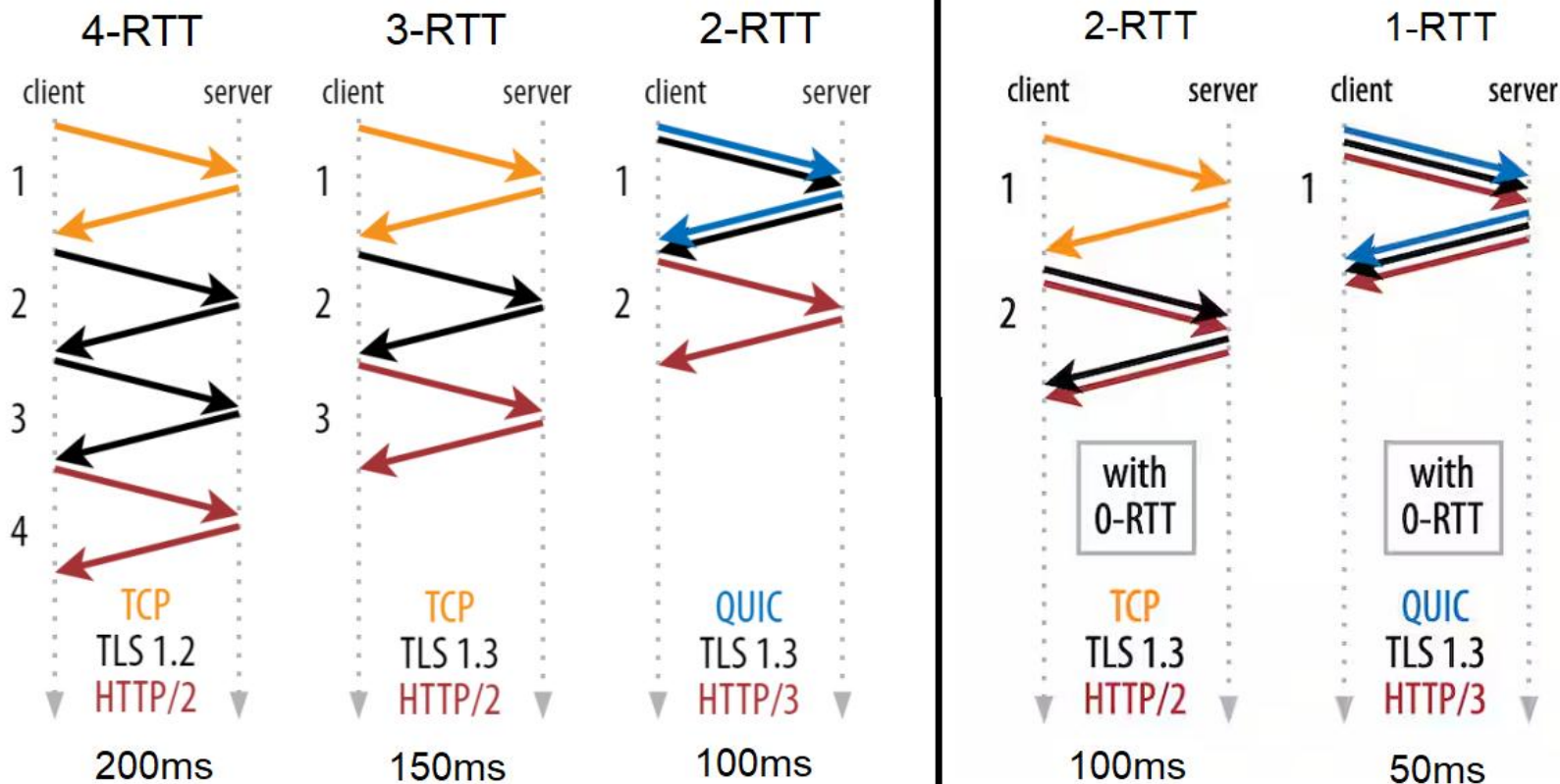
Wire Image of TCP/TLS vs QUIC



Practical implication: A middlebox or packet capture will have much less visibility to troubleshoot layer-4 issues, unless decryption is used



RTTs of TCP/TLS vs QUIC



Example calculations for average RTT latency of 50ms

RTT of TCP/TLS vs QUIC



“Here’s my microsecond: 984 feet. Sometimes I think we out to hang one over every programmer’s desk – or maybe around their neck! – so they know what they’re throwing away when they throw away a microsecond.”

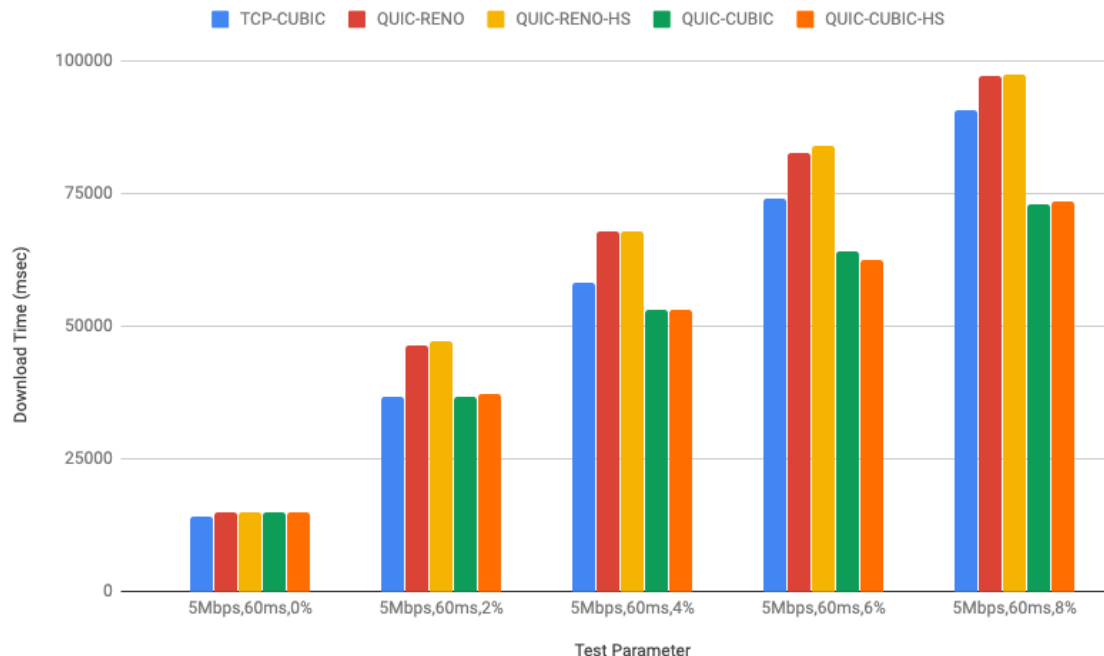
~Rear Admiral Grace Hopper

Imagine telling Admiral Hopper about how frequently network engineers throw away milliseconds! (983,571 feet)

https://youtu.be/gYqF6-h9Cvg?si=wU3t2RfUJ_lj_H7l

Congestion Control in QUIC

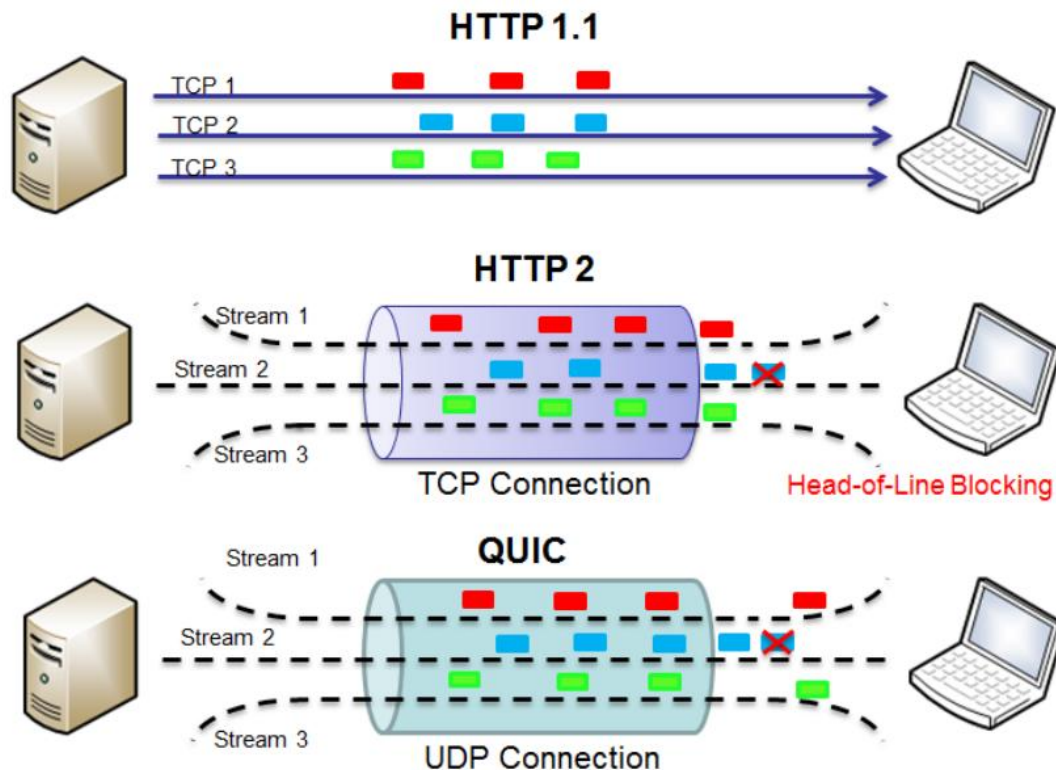
- User-Space congestion algorithms
 - Reno, CUBIC, Hystart, etc
- Faster deployment of improvements
- Independent from OS TCP stack
- Per-connection congestion feedback
- Better handling of packet loss
- Reduced latency during congestion



Ref: <https://blog.cloudflare.com/cubic-and-hystart-support-in-quiche/>

Multiplexing in QUIC

- Multiple streams per connection
- Streams operate independently
- No cross-stream Head-of-Line blocking
- Packet loss affects single stream
- Faster recovery than TCP
- Improved latency under congestion
- Ideal for modern web apps



Common QUIC Myths

1. Myth: “QUIC is proprietary, with proprietary encryption”
 - Fact: QUIC is an IETF standard, with TLS standard encryption
2. Myth: “QUIC is only used for HTTP/3”
 - Fact: QUIC is a general-purpose transport-layer protocol for any application
3. Myth: “QUIC is a niche protocol with low adoption”
 - Fact: QUIC adoption is accelerating extremely rapidly

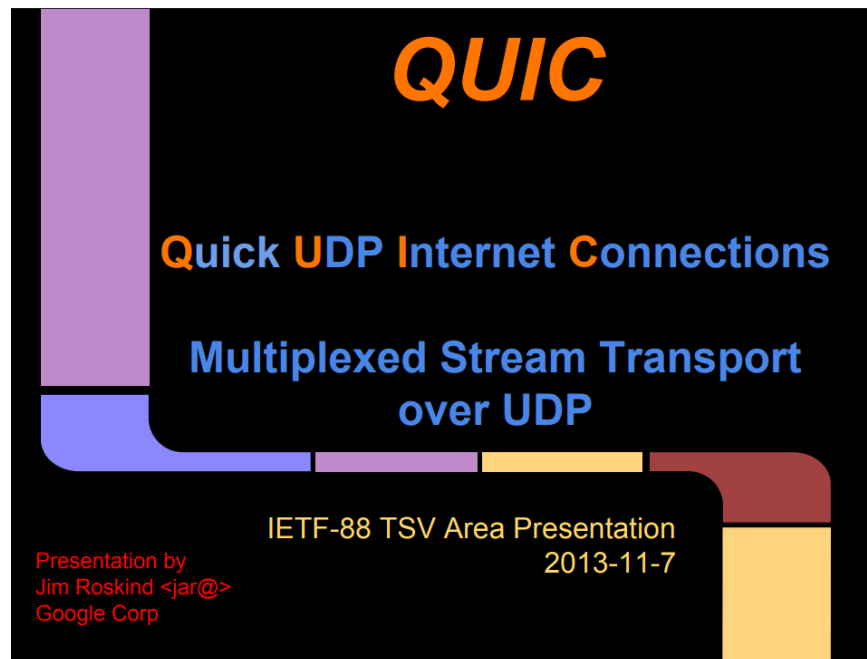
Who developed QUIC and when?

- Jim Roskind started development in 2012
 - With Google at the time
 - Origin of the “Google proprietary” myth
 - He now works for Amazon
- Jim presented his early efforts at IETF in 2013
- First informational draft in published 2015
- IETF QUIC Working Group officially established in 2016
 - RFC 8999-9002 reached “proposed standard” status in 2021



“Quick UDP Internet Connections”

- Claim: QUIC is not an acronym; the RFCs never spell it out
- However, Roskind uses “Quick UDP Internet Connections” in his earliest IETF presentations
- He still uses it [today](#)



Notable QUIC Implementations

Name (Author)	Language	License
<u>QUICHE</u> (Google)	C++	BSD-3-Clause License
<u>QUICHE</u> (Cloudflare)	Rust	BSD-2-Clause License
<u>Nego</u> (Mozilla)	Rust	Apache License 2.0 / MIT License
<u>MsQuic</u> (Microsoft)	C	MIT License
<u>s2n-quic</u> (Amazon)	Rust	Apache-2.0 license
<u>mfst</u> (Meta)	C++	MIT License
<u>lsquic</u> (LiteSpeed)	C	MIT License
<u>quicly</u> (H2O)	C	MIT License
<u>quinn</u> (quinn-rs)	Rust	MIT License

More examples: https://en.wikipedia.org/wiki/QUIC#Source_code

QUIC? HTTP/3? What's the difference?

- Discussions about QUIC often revolve around HTTP/3 (RFC 9114)
- HTTP/3 is the most common app-layer protocol to use QUIC.
- But it's not the only one!
- In other words, HTTP/3 always rides on QUIC, but QUIC doesn't always carry HTTP/3
- Let's look at some other applications that use QUIC

Other L7 Protocols That Use QUIC

Most L7 protocols on QUIC use destination port UDP/443, relying on the ALPN in the TLS Client Hello to distinguish between services:

- SMB-over-QUIC
 - Default SMB version in Windows Server 2022 and most Azure services
- MASQUE (New tunneling/VPN mechanism)
 - Multiplexed Application Substrate over QUIC Encryption ([RFC 9484](#))
- Media-over-QUIC (MoQ)
 - [Draft Stage](#)
- [XMPP-over-QUIC](#)
 - Extensible Messaging and Presence Protocol

Other L7 Protocols That Use QUIC

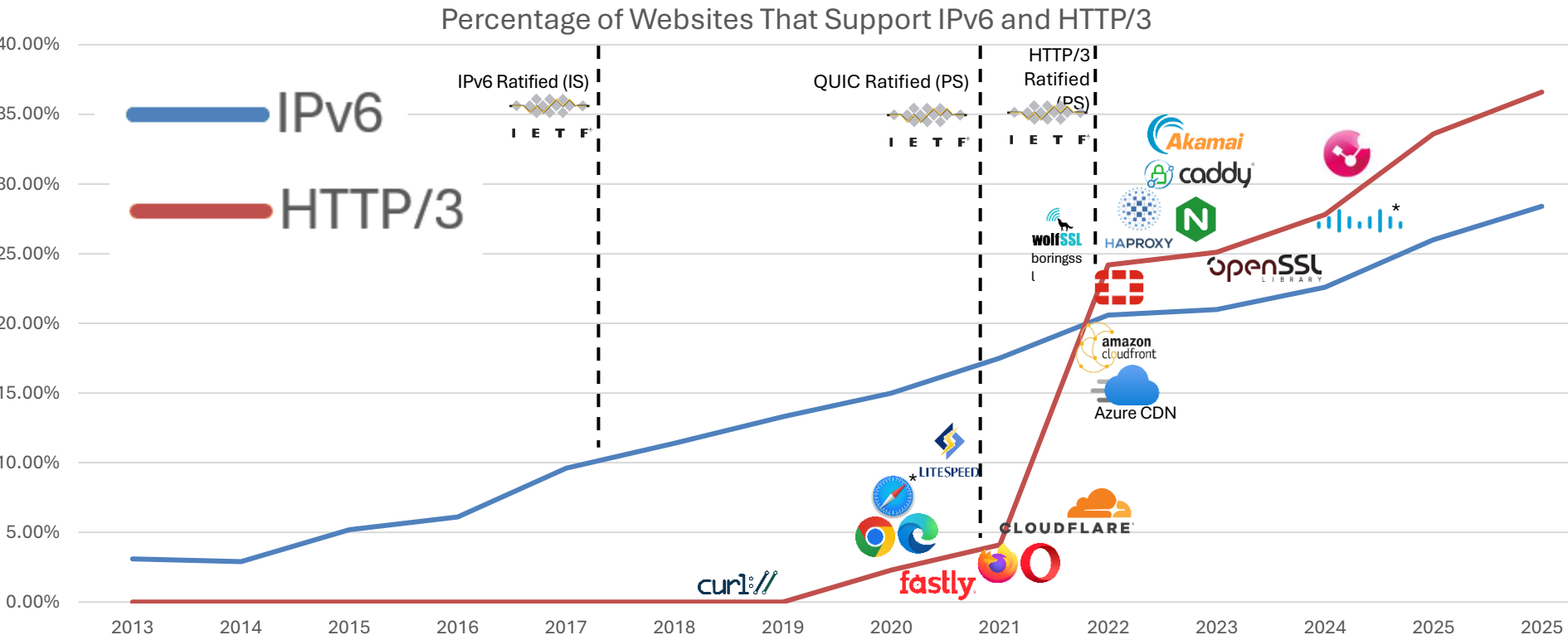
Some L7 protocols use a destination port OTHER than UDP/443:

- BGP-over-QUIC
 - [Draft Stage](#)
 - Default destination port is UDP/179
- DNS-over-QUIC (DoQ)
 - [RFC 9520](#)
 - Default destination port is UDP/853
- **Practical implication:** Merely blocking destination port UDP/443 doesn't necessarily block all QUIC traffic!
- Remember: threat actors can (and will!) use alternate port numbers!

How rapidly is QUIC being adopted?

- The RFCs for QUIC reached “proposed standard” status in 2021
 - By 2025, over 37% of all websites are using HTTP/3 on QUIC
- IPv6 reached “proposed standard” status in 1995
 - Only 28% of all websites are using IPv6 as of 2025
- QUIC adoption in 5 years >> IPv6 adoption in 31 years
 - QUIC is backwards compatible with the existing internet infrastructure
 - IPv6 requires full dual-stack, or 4-to-6 NAT gateways.

Server-Side Adoption of HTTP/3 over Time

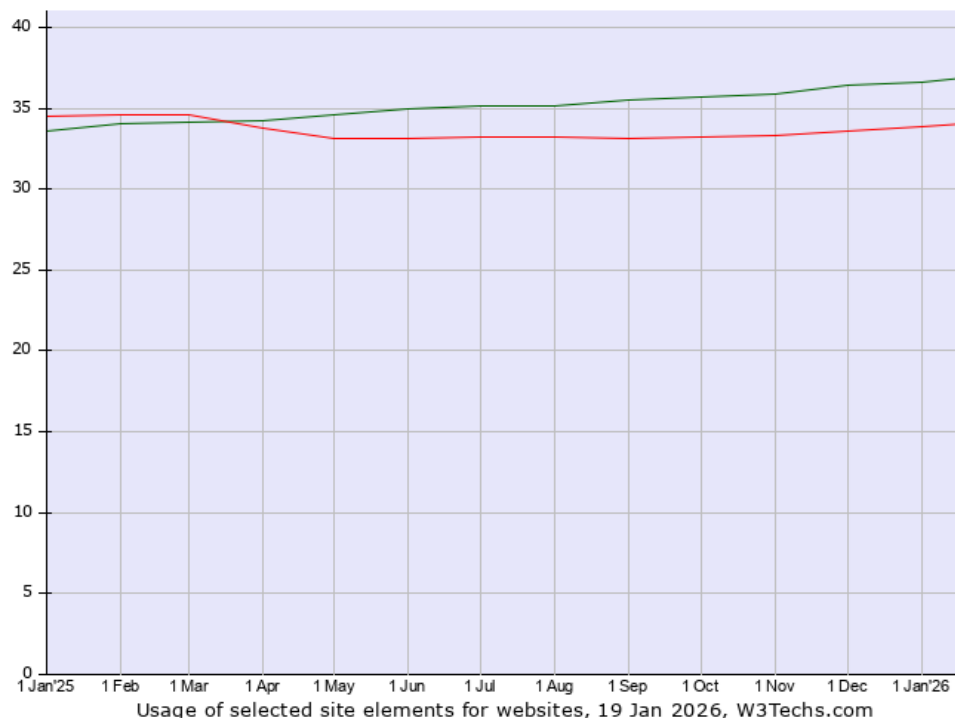


HTTP/3 vs HTTP/2 Adoption by Top Sites

Historical trend

This diagram shows the historical trend in the percentage of websites using the selected technologies.

Our dedicated trend survey shows more [site elements usage and market share trends](#).



Usage of selected site elements for websites, 19 Jan 2026, W3Techs.com

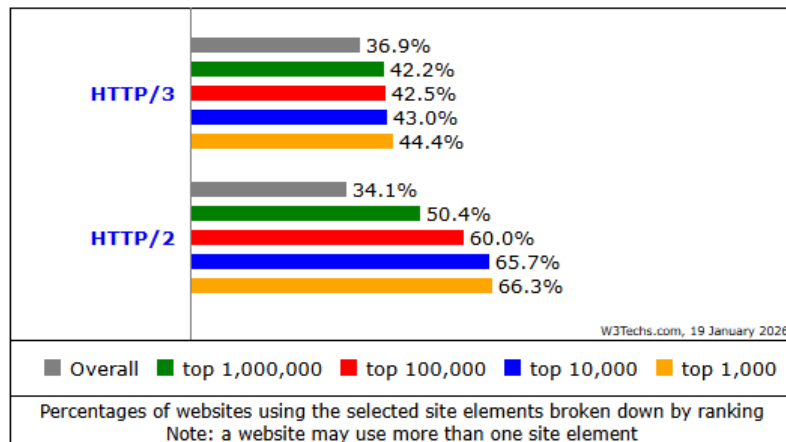
Usage broken down by ranking

This diagram shows the percentages of websites using the selected technologies broken down by ranking.

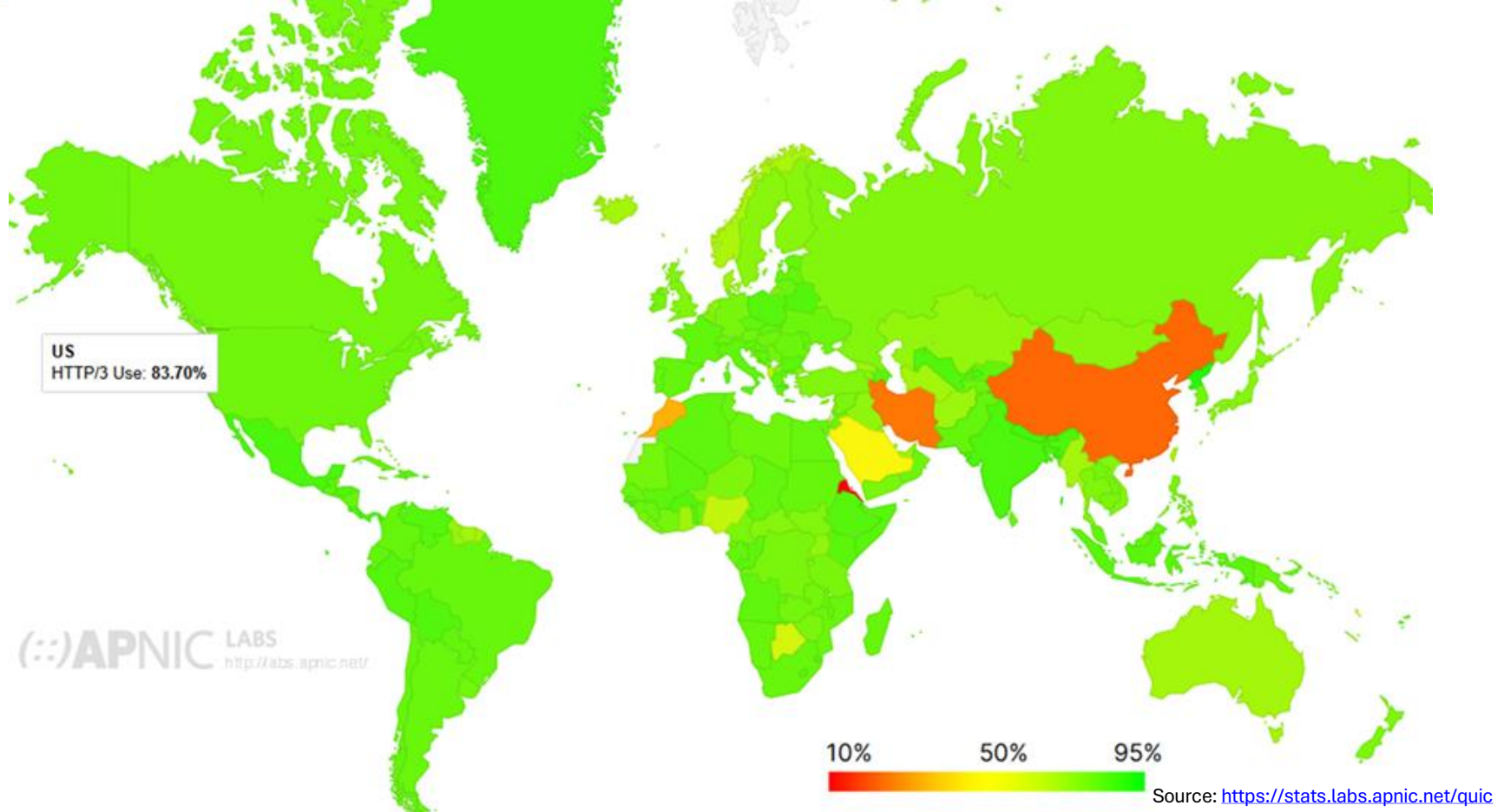
How to read the diagram:

HTTP/3 is used by 36.9% of all the websites.

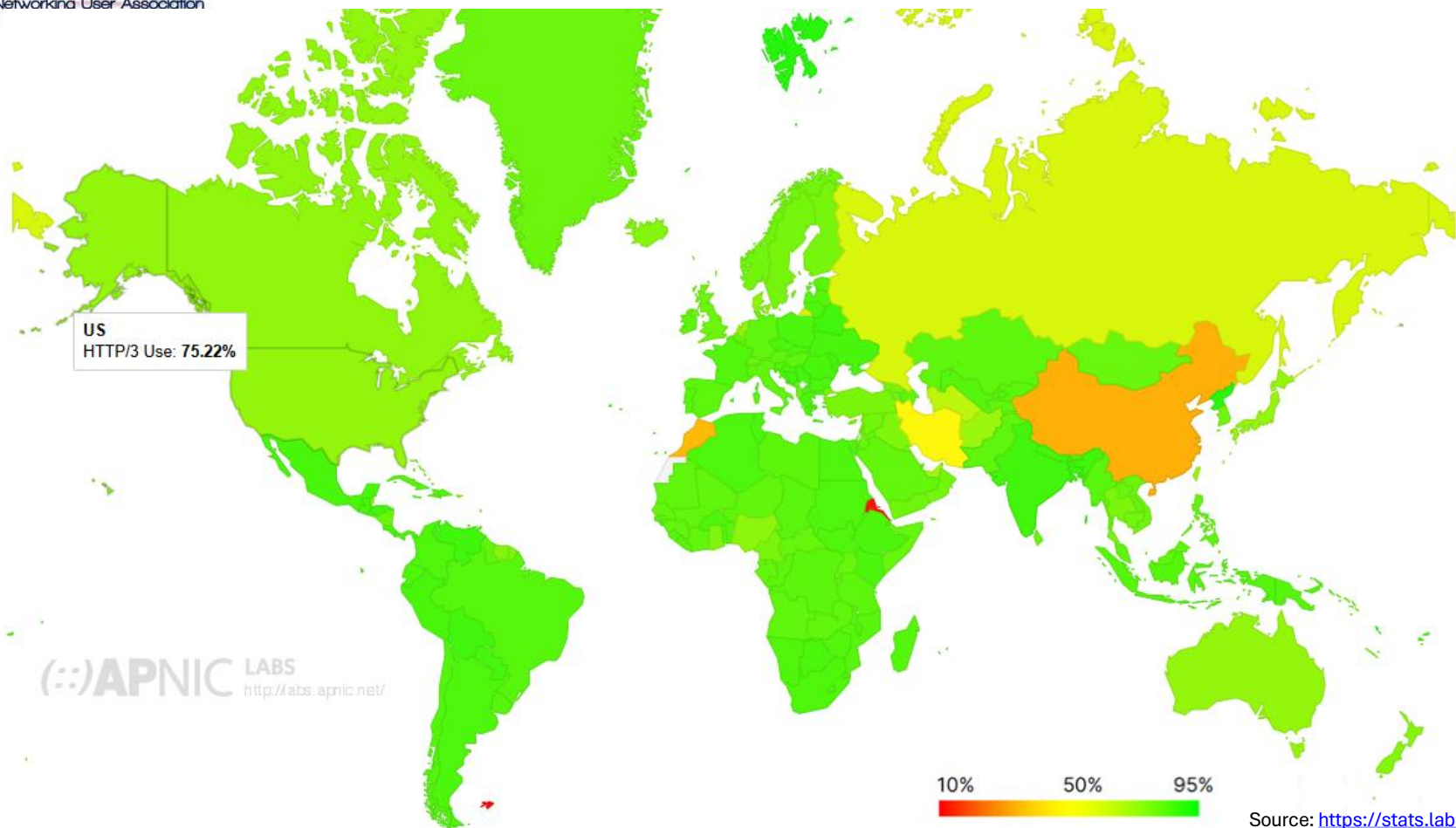
HTTP/3 is used by 42.2% of all the websites that rank in the top 1,000,000.



Client-Side Adoption of HTTP/3 >80% in Most of the World – 2024



Client-Side Adoption of HTTP/3 >80% in Most of the World – 2025



Source: <https://stats.labs.apnic.net/quic>

QUIC is Now – QUIC is the Future

- My prediction: QUIC will largely replace TCP as THE dominate transport protocol on the internet within the next 10 years
- You might never need to learn IPv6 in your lifetime
- But you will definitely need to learn QUIC

What does this mean for network operations?

- Layer-3 and below remains unchanged
 - Route/switch/wireless
- Layer-4 and above is affected
 - Servers/Clients/Middleboxes
- Current network tooling was originally designed for a TCP world
- Updates will be necessary to implement QUIC

What does this mean for network operations?

- Updates will be necessary to implement QUIC
- This includes everything from analytics tools to middleboxes
 - Wireshark: Must import decryption keys to see anything meaningful
 - Powershell Test-NetConnection: Can't check a UDP socket!
 - Try curl --http3-only instead
 - Nmap: requires some [custom modules](#)
 - Middleboxes will need updates from OEMs:
 - NGFWs, WAFs, load balancers, SD-WAN routers, VPNs, SSE/SASE, etc.

What does this mean for network security?

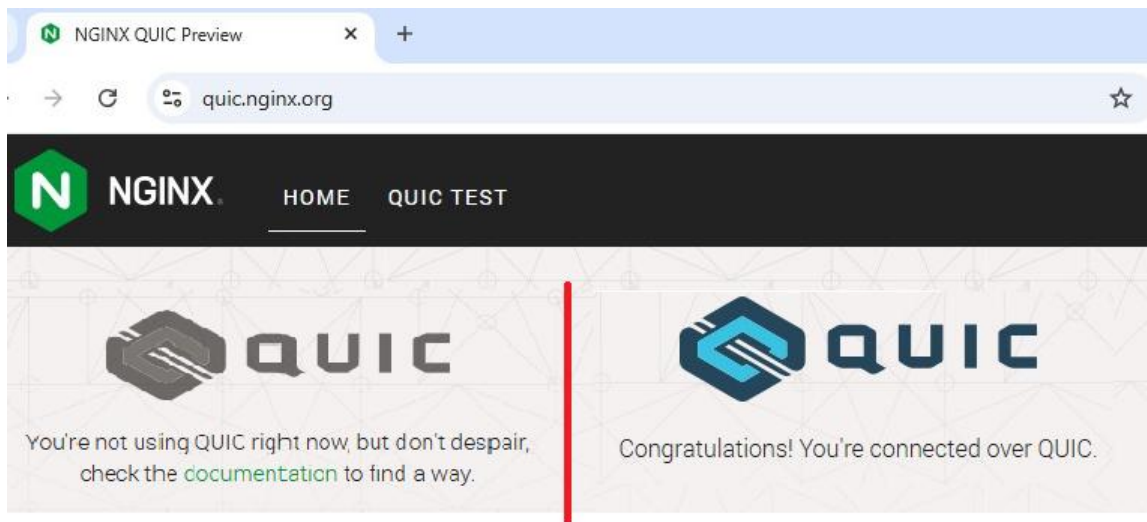
- NetSec OEM implementations of QUIC inspection are developing
- Historically, recommendation was to block QUIC and force downgrade to TCP+TLS
 - Without QUIC implementation, security features can't function
 - Web/URL Filtering, Application Control, IPS, etc.
- Degrades the user experience and hinders QUIC adoption
- Allowing AND inspecting QUIC/HTTP3 is the best of both worlds
 - But this requires firmware updates

What does this mean for App Devs?

- All mainstream browsers already support HTTP/3 by default
- Most CDNs do too
- Load-balancer implementations are still developing
 - Front-end support is catching up
 - Back-end support is still lagging
- Practically speaking, front-end support is the most impactful

NGINX QUIC Test Site for Client-Side Tests: <https://quic.nginx.org>

Easy way to test if you can use QUIC or not, including with Deep SSL Inspection



Welcome to the demo site for QUIC+HTTP/3 support in nginx

Certificate Viewer: quic.nginx.org

General Details

Issued To

Common Name (CN)	quic.nginx.org
Organization (O)	<Not Part Of Certificate>
Organizational Unit (OU)	<Not Part Of Certificate>

Issued By

Common Name (CN)	FGT70FTK22000045
Organization (O)	Fortinet
Organizational Unit (OU)	Certificate Authority

Validity Period

Issued On	Tuesday, May 20, 2025 at 1:26:40 AM
Expires On	Monday, August 18, 2025 at 1:26:39 AM

SHA-256 Fingerprints

Server-Side Testing: <https://http3check.net/>

Easy way to test if a given website supports HTTP3



Powered By LiteSpeed

STANDARD

ADVANCED

ABOUT

google.com

CHECK

google.com

✓ QUIC is supported

✓ HTTP/3 is supported

HTTP/3 Check established a QUIC connection for all attempts made with the given endpoint. See the metrics below for more information.

0-RTT H3 H3-29

CONNECTION ID ⓘ	PACKET RX ⓘ	HANDSHAKE DONE ⓘ
BCA3212ABC...	1.779	11.615
2AFB79179E...	11.914	12.215

Read [detailed descriptions](#) of the items and metrics shown above.

HTTP Header

HTTP/1.1 301 Moved Permanently
location: <https://www.google.com/>
content-type: text/html; charset=UTF-8
content-security-policy-report-only: object-src 'none';base-uri 'self';script-src 'nonce-

QLog Output ⓘ

```
{
  "qlog_version": "0.1",
  "description": "google.com",
  "connections": [
    {
      "qlog_version": "0.1",
      "vantagepoint": "NETWORK",
      "connectionid": "BCA3212ABC68B270",
      "starttime": 43866589443100,
      "fields": [
        "time",
        "category",
        "type",
        "trigger",
        "data"
      ],
      "events": [
        [
          0,
          "CONNECTIVITY",
          "NEW_CONNECTION",
          "LINE",
          {
            "ip_version": "6",
            "ip_address": "142.250.190.100",
            "port": 443,
            "protocol": "QUIC",
            "type": "client",
            "version": "0x0000000000000000"
          }
        ]
      ]
    }
  ]
}
```

Chrome Native Tools: Net Export and Netlog Viewer

<chrome://net-export> and <https://netlog-viewer.appspot.com/#import>

chrome-net-export-log.json

Import

Proxy

Events

Timeline

DNS

Sockets

Alt-Svc

HTTP/2

QUIC

Reporting

Cache

Modules

Prerender

QUIC Option	Value
Supported Versions	RFCv1
Connection options	
Max Packet Length	1250
Idle Connection Timeout In Seconds	30
Reduced Ping Timeout In Seconds	15
Packet Reader Yield After Duration in Milliseconds	
Mark QUIC Broken When Network Blackholes	false
Do Not Mark QUIC Broken on Network Changes	false
Retry without Alt-Svc on QUIC Errors	true
Do Not Fragment	false
Allow Server Migrations	true
Migrate Sessions Early V2	false
Migrate Sessions on Network Change V2	false
Retransmittable on Wire Timeout in Milliseconds	true
Disable Bidirectional Streams	false
Race Cert Verification	false
Race Stale DNS On Connection	false
Estimate Initial RTT	false
Force Head of Line Blocking	false
Max Server Configs Stored in Properties	0
Origins To Force QUIC On	
Server Push Cancellation	false

QUIC sessions

View live QUIC sessions

Host	Version	Peer address	Connection ID	Active stream c
accounts.youtube.com:443	RFCv1	172.217.4.78:443	e8680a9f0d59ff16	0
www.youtube.com:443	RFCv1	142.251.179.93:443	f3007b2b5b9680bb	0
youtube.com:443	RFCv1	142.251.111.91:443	e6e0ec8fa0fab730	0

Total stream count	Packets Sent	Packets Lost	Packets Received	Connected
1	13	0	16	true
21	136	0	295	true
1	13	0	16	true

chrome-net-export-log.json

Import

Proxy

Events

Timeline

DNS

Sockets

Alt-Svc

HTTP/2

QUIC

Reporting

Cache

Modules

Prerender

(?) type:QUIC_SESSION f3007b2b5b9680bb 1 of 660

ID	Source	Type	Description
241002	QUIC_SESSION		

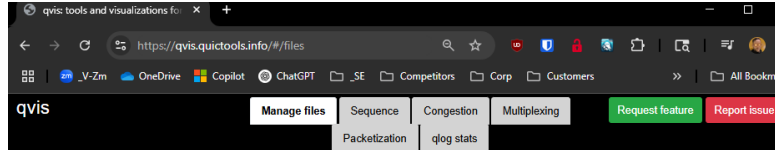
241002: QUIC_SESSION

Start Time: 2025-06-09 15:09:35.287

t=32295 [st=12870] QUIC_SESSION_POOL_ATTACH_HTTP_STREAM_JOB_TO_EXISTING_SESSION
--> source_dependency = 241214 (HTTP_STREAM_JOB)
t=32295 [st=12870] HTTP3_PRIORITY_UPDATE_SENT
--> prioritized_element_id = 4
--> priority_field_value = "u=0, i"
t=32295 [st=12870] QUIC_SESSION_STREAM_FRAME_SENT
--> fin = false
--> length = 12
--> offset = 51
--> stream_id = 2
t=32295 [st=12870] QUIC_CHROMIUM_CLIENT_STREAM_SEND_REQUEST_HEADERS
--> :method: GET
:authority: www.youtube.com
:scheme: https
:path: /
upgrade-insecure-requests: 1
user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) C
accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng
x-browser-channel: stable
x-browser-year: 2025
x-browser-validation: J90SnQ4ykHeik2f8jbqHhIrks=
x-browser-copyright: Copyright 2025 Google LLC. All rights reserved.
sec-ch-ua: "Google Chrome";v="137", "Chromium";v="137", "Not(A)Brand";v="24"
sec-ch-ua-mobile: ?0
sec-ch-ua-full-version: "137.0.7151.69"
sec-ch-ua-arch: "x86"
sec-ch-ua-form-factors: "Desktop"
sec-ch-ua-platform: "Windows"
sec-ch-ua-platform-version: "10.0.0"
sec-ch-ua-model: ""
sec-ch-ua-bitness: "64"
sec-ch-ua-wow64: ?0
sec-ch-ua-full-version-list: "Google Chrome";v="137.0.7151.69", "Chromium";v="137.0.7151.69",
sec-ch-ua-form-factors: "Desktop"
service-worker-navigation-preload: true
x-client-data: CI+2yQEIorbJAQipncoBCKPcygEIIaHLAQiRo8sBCIwzQEIEo30AQjk7c4BCMXvzgEI4fDOARjh4s4
sec-fetch-site: none
sec-fetch-mode: navigate
sec-fetch-user: ?1

qlog and qvis: https://qvis.quictools.info

QUIC Logging in JSON format and visualization tool



Welcome to qvis v0.1, the QUIC and HTTP/3 visualization toolsuite!

To be able to visualize something, you need to load some data. We have several options for that:

Option 1 Load a file by URL

You can load .qlog, .sqlog, .netlog, .pcap (alongside separate .keys) and .pcapng (with embedded keys) files. You can also load a .json file that lists several other files to be fetched (for the format, see [the pcap2qlog documentation](#). Or try [an example](#)).

If you're looking for inspiration, [quant](#) has public qlogs, as does [aioquic](#). [QUIC Tracker](#) provides .pcap files for all its tests and has a convenient integration with qvis from its UI. Many of the tests in the [QUIC Interop Runner](#) also include .qlog and .pcap output.

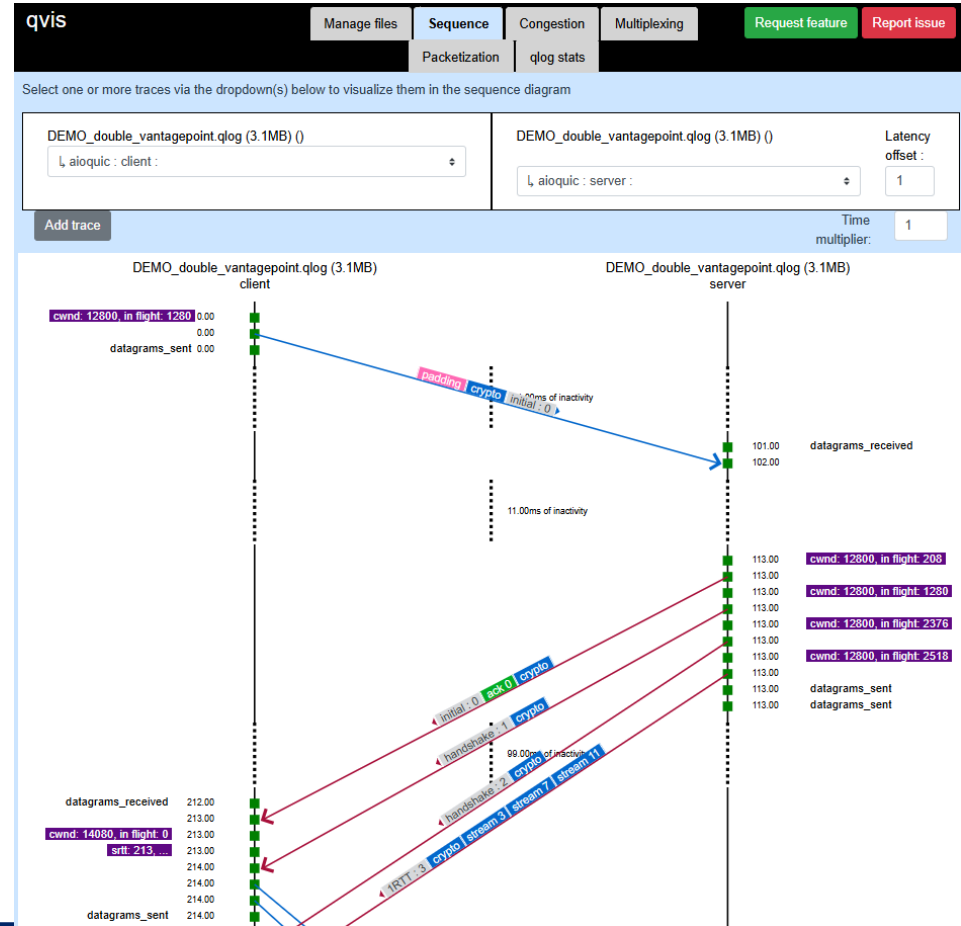
Option 2 Upload a file

Upload currently supports .qlog, .sqlog, .json, and .netlog files. No data is transferred to the server. Eventually we will also support .pcap, .pcapng and .qtr files.

Note: Chrome netlog must be explicitly given the .netlog extension before uploading to qvis.

Option 3 Load some premade demo files

This will load a few example files that you can visualize to get an idea of what's possible



Videos

- [Jim Roskind Explaining QUIC Development History at AWS re:Invent 2022](#)
- [Explaining QUIC: the protocol that is both very similar to and very different from TCP By Peter Door](#)
- [How the QUIC Handshake Works by Chris Greer of Wireshark Talks](#)
- [Will QUIC Kill TCP? by Chris Greer of Wireshark Talk](#)
- [Analyzing QUIC and HTTP/3 traffic with qlog and qvis](#)
- [BGP over QUIC](#)
- [Decoding QUIC](#)
- [FortiOS 7.2.0 Demo of QUIC Inspection](#)

Slide Decks

- [Jim Roskind's Initial QUIC Presentation Deck at IETF-88 in 2013](#)

Articles

- [QUIC is Not a TCP Replacement](#) by Bruce Davie
- [It's TCP vs. RPC All Over Again](#) by Larry Peterson
- [TCP: The "P" is for Platform](#) by Larry Peterson
- [The Importance of Thinking Across Layer Boundaries](#) by Bruce Davie
- [Comparing TCP and QUIC](#) by Geoff Huston
- [A QUIC progress report](#) by Geoff Huston

Informational RFCs

- [RFC 1263: TCP Extensions Considered Harmful](#)

Ideas for Future Talks – Let me know what you're interested in!

More Developments in Protocol Land

- Encrypted Client Hello (ECH)
- HTTP/SVCB Records in DNS
- Post-Quantum Ciphers (PQC) in TLS
- UDP-Lite
- QUIC Version 2

Questions, Comments, Feedback

- Email: vweis@fortinet.com
- Github: <https://github.com/weis-victor>
- LinkedIn: <https://www.linkedin.com/in/victor-weis/>



Favorite Aphorisms – The Pith of Life



- All models are wrong; some are useful.
- There are no solutions — only trade-offs.
- Automation without understanding is just doing stupid things faster.
- To work less, you must learn more.
- Every happy network is happy in the same way; every unhappy network is unhappy in a different way.